



VANLANG UNIVERSITY
School of Technology
Information Technology



HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

(Database Management System)

CHƯƠNG 10: GIAO TÁC VÀ TRUY XUẤT ĐỒNG THỜI



GVGD: THS. NGUYỄN THỊ QUYÊN



HỌC KỲ II - NĂM HỌC 2024-2025



KHÓA 29T-IT





NỘI DUNG

- 01.** GIAO TÁC LÀ GÌ
- 02.** TÍNH CHẤT CỦA GIAO TÁC
- 03.** MÔ HÌNH GIAO TÁC TRONG SQL
- 04.** TRUY XUẤT ĐỒNG THỜI



NGOÀI LỀ (1/4)

- GIAO DỊCH NGÂN HÀNG: Chuyển tiền từ A sang B

Sự cố

Update TAIKHOAN set **SoDu=SoDu-50** where MATK=A

Update TAIKHOAN set **SoDu=SoDu+50** where MATK=B

Lỗi: Có thể đã rút tiền từ A nhưng chưa cập nhật được số dư của B



NGOÀI LỀ (2/4)

- GIAO DỊCH NGÂN HÀNG: Nhiều người cùng rút tiền từ A, ví dụ số dư của A là 100, có 3 người rút số tiền là 70, 80, 90:

Đồng
thời

Update TAIKHOAN set **SoDu=SoDu-70** where MATK=A

Update TAIKHOAN set **SoDu=SoDu-80** where MATK=A

Update TAIKHOAN set **SoDu=SoDu-90** where MATK=A

Lỗi: Có thể rút được tiền nhiều hơn số tiền thực có.



NGOÀI LỀ (3/4)

- GIAO DỊCH ĐẶT CHỖ: Nhiều người xem phim cùng đặt 1 ghế

Đồng
thời

Update GHE set MaVe=**001** where MAGHE='G7'

Update GHE set MaVe=**002** where MAGHE='G7'

Update GHE set MaVe=**003** where MAGHE='G7'

Lỗi: nhiều khán giả đặt được cùng 1 ghế



NGOÀI LỀ (4/4)

- ĐÁNH GIÁ:
- Thường xuyên xảy ra vấn đề cần phải **nhất quán** dữ liệu nếu một xử lý gặp **sự cố** hoặc khi các xử lý được gọi truy xuất **đồng thời**.
- Giao tác là một khái niệm nền tảng của điều khiển truy xuất **đồng thời** và khôi phục khi có **sự cố**.



0. GIAO TÁC LÀ GÌ?

- **Giao tác** (hay còn gọi là **Giao dịch, Transaction**) là một đơn vị xử lý **nguyên tố** gồm một chuỗi các hành động **đọc/ghi (read/write)** trên các **đối tượng cơ sở dữ liệu**.

T

- Statement 1
- Statement 2
- Statement 3
- Statement 4
-



1. TÍNH CHẤT CỦA GIAO TÁC

- Các giao dịch có **bốn** tính chất chuẩn sau đây, thường được gọi bằng từ viết tắt **ACID**.
 - Tính nguyên tố (Atomicity).
 - Tính nhất quán (Consistency).
 - Cô lập (Isolation).
 - Tính bền vững (Durability)



1. TÍNH CHẤT CỦA GIAO TÁC

- Các giao dịch có **bốn** tính chất chuẩn sau đây, thường được gọi bằng từ viết tắt **ACID**.
 - **Nguyên tử (Atomicity)**
 - Hoặc là toàn bộ hoạt động của giao dịch được phản ánh đúng đắn trong CSDL hoặc không có hoạt động nào cả
 - **Nhất quán (Consistency)**
 - Một giao tác được thực hiện độc lập với các giao tác khác xử lý đồng thời với nó để bảo đảm tính nhất quán cho CSDL



1. TÍNH CHẤT CỦA GIAO TÁC

- Các giao dịch có **bốn** tính chất chuẩn sau đây, thường được gọi bằng từ viết tắt **ACID**.
 - **Cô lập (Isolation)**
 - Một giao tác không quan tâm đến các giao tác khác xử lý đồng thời với nó.
 - **Bền vững (Durability)**
 - Mọi thay đổi mà giao tác thực hiện trên CSDL phải được ghi nhận bền vững.



1. TÍNH CHẤT CỦA GIAO TÁC

- Ví dụ: $A=100$, $B=200$ ($A+B=300$), A chuyển cho B 50.

```
T:   Read (A, t) ;  
      t:=t-50 ;  
      Write (A, t) ;  
      Read (B, t) ;  
      t:=t+50 ;  
      Write (B, t) ;
```

- **Nhất quán (Consistency):**
 - Tổng $A+B$ là không đổi (**$A=50$, $B=250$, $A+B=300$**)
 - Nếu CSDL nhất quán trước khi T được thực hiện thì sau khi T hoàn tất CSDL vẫn còn nhất quán



1. TÍNH CHẤT CỦA GIAO TÁC

- Ví dụ: $A=100$, $B=200$ ($A+B=300$), A chuyển cho B 50.

T: Read (A, t) ;

$t := t - 50$;

—————→ Write (A, t) ;

 Read (B, t) ;

$t := t + 50$;

—————→ Write (B, t) ;

- Nguyên tố (Atomicity):
 - $A=100$, $B=200$ ($A+B=300$)
 - Tại thời điểm sau khi Write(A,t)
 $A=50$, $B=200$ ($A+B=250$) - CSDL không nhất quán
 - Tại thời điểm sau khi Write(B,t)
 $A=50$, $B=250$ ($A+B=300$) - CSDL nhất quán



1. TÍNH CHẤT CỦA GIAO TÁC

- Ví dụ: $A=100$, $B=200$ ($A+B=300$), A chuyển cho B 50.

```
T:   Read (A, t) ;  
      t:=t-50 ;  
      Write (A, t) ;  
      Read (B, t) ;  
      t:=t+50 ;  
      Write (B, t) ;
```

- **Bền vững (Durability):**
 - Khi T kết thúc thành công.
 - Dữ liệu sẽ không thể nào bị mất bất chấp có sự cố hệ thống xảy ra.



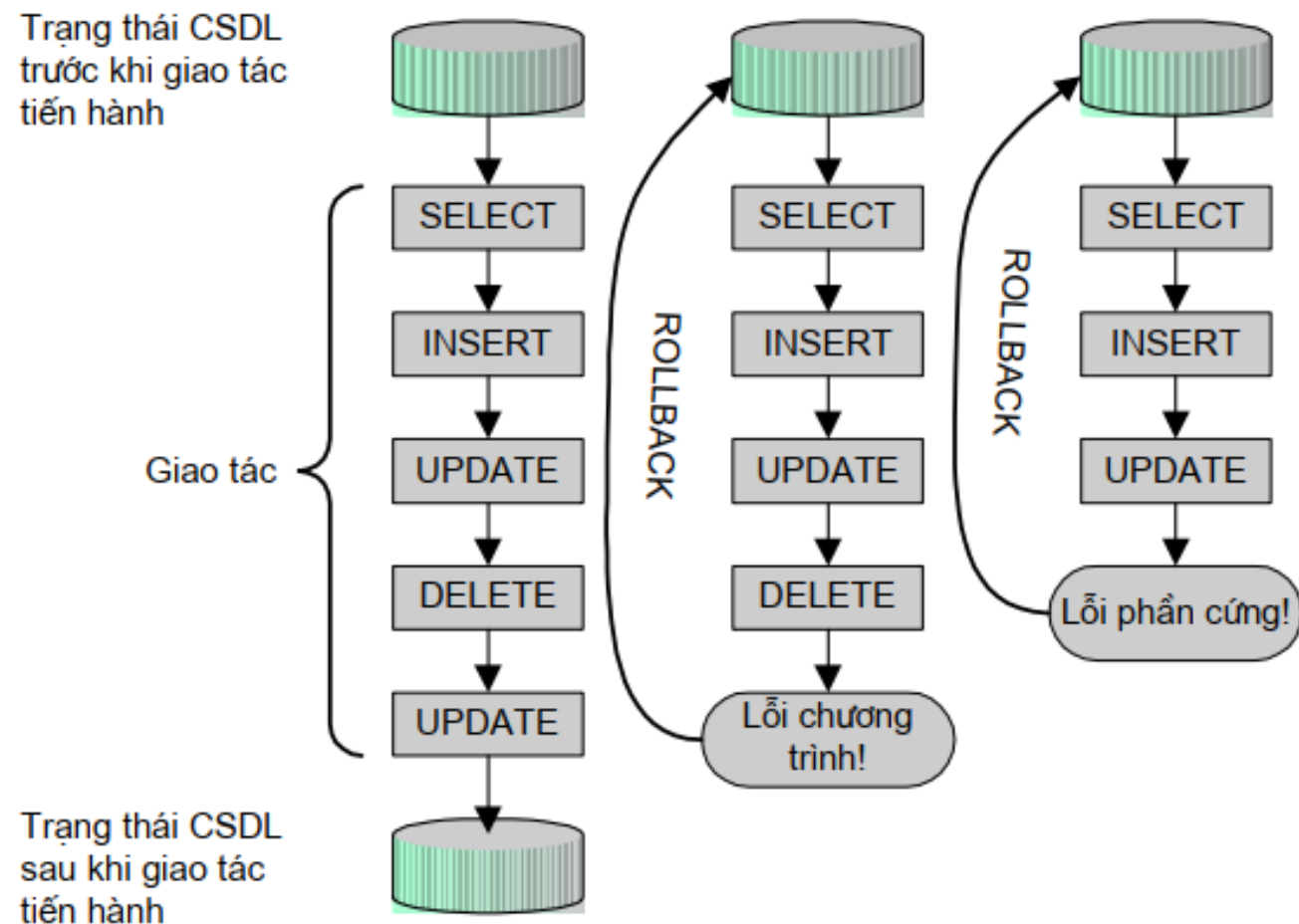
1. TÍNH CHẤT CỦA GIAO TÁC

- Ví dụ: $A=100$, $B=200$ ($A+B=300$), A chuyển cho B 50.

T : Read (A, t) ;
 $t := t - 50$;
T' $\longrightarrow$ Write (A, t) ;
 Read (B, t) ;
 $t := t + 50$;
 Write (B, t) ;

- **Cô lập (Isolation):**
 - Giả sử có 1 giao tác T' thực hiện phép toán $A+B$ và chen vào giữa thời gian thực hiện của T.
 - T' kết thúc: $A+B=50+200=250$
 - T kết thúc: $A+B=50+250=300$
 - Hệ thống của các giao tác thực hiện đồng thời có trạng thái tương đương với trạng thái hệ thống của các giao tác thực hiện tuần tự theo 1 thứ tự nào đó.

1. TÍNH CHẤT CỦA GIAO TÁC



Giao tác trên SQL



2. MÔ HÌNH GIAO TÁC TRONG SQL

- Giao tác SQL được định nghĩa dựa trên các câu lệnh xử lý giao tác sau đây:
 - **BEGIN TRANSACTION:** Bắt đầu một giao tác
 - **SAVE TRANSACTION:** Đánh dấu một vị trí trong giao tác (gọi là điểm đánh dấu).
 - **ROLLBACK TRANSACTION:** Quay lui trở lại đầu giao tác hoặc một điểm đánh dấu trước đó trong giao tác.



2. MÔ HÌNH GIAO TÁC TRONG SQL

- **COMMIT TRANSACTION:** Đánh dấu điểm kết thúc một giao tác. Khi câu lệnh này thực thi cũng có nghĩa là giao tác đã thực hiện thành công.
- **ROLLBACK [WORK]:** Quay lui trở lại đầu giao tác.
- **COMMIT [WORK]:** Đánh dấu kết thúc giao tác.
- Câu lệnh này đánh dấu điểm bắt đầu của một giao tác và có cú pháp như sau:

BEGIN TRANSACTION [tên_giao_tác]



2. MÔ HÌNH GIAO TÁC TRONG SQL

- **Ví dụ 1:** Giao tác dưới đây kết thúc do lệnh **ROLLBACK TRANSACTION** và mọi thay đổi về mặt dữ liệu mà giao tác đã thực hiện (**UPDATE**) đều không có tác dụng.

BEGIN TRANSACTION GIAOTAC1

UPDATE monhoc SET sodvht=5 WHERE sodvht=3

UPDATE diemthi SET diemlan2=0 WHERE diemlan2 IS NULL

ROLLBACK TRANSACTION GIAOTAC1



2. MÔ HÌNH GIAO TÁC TRONG SQL

- **Ví dụ 2:** Giao tác dưới đây kết thúc do lệnh **COMMIT** và mọi thay đổi về mặt dữ liệu mà giao tác đã thực hiện (**UPDATE**) đều có tác dụng.

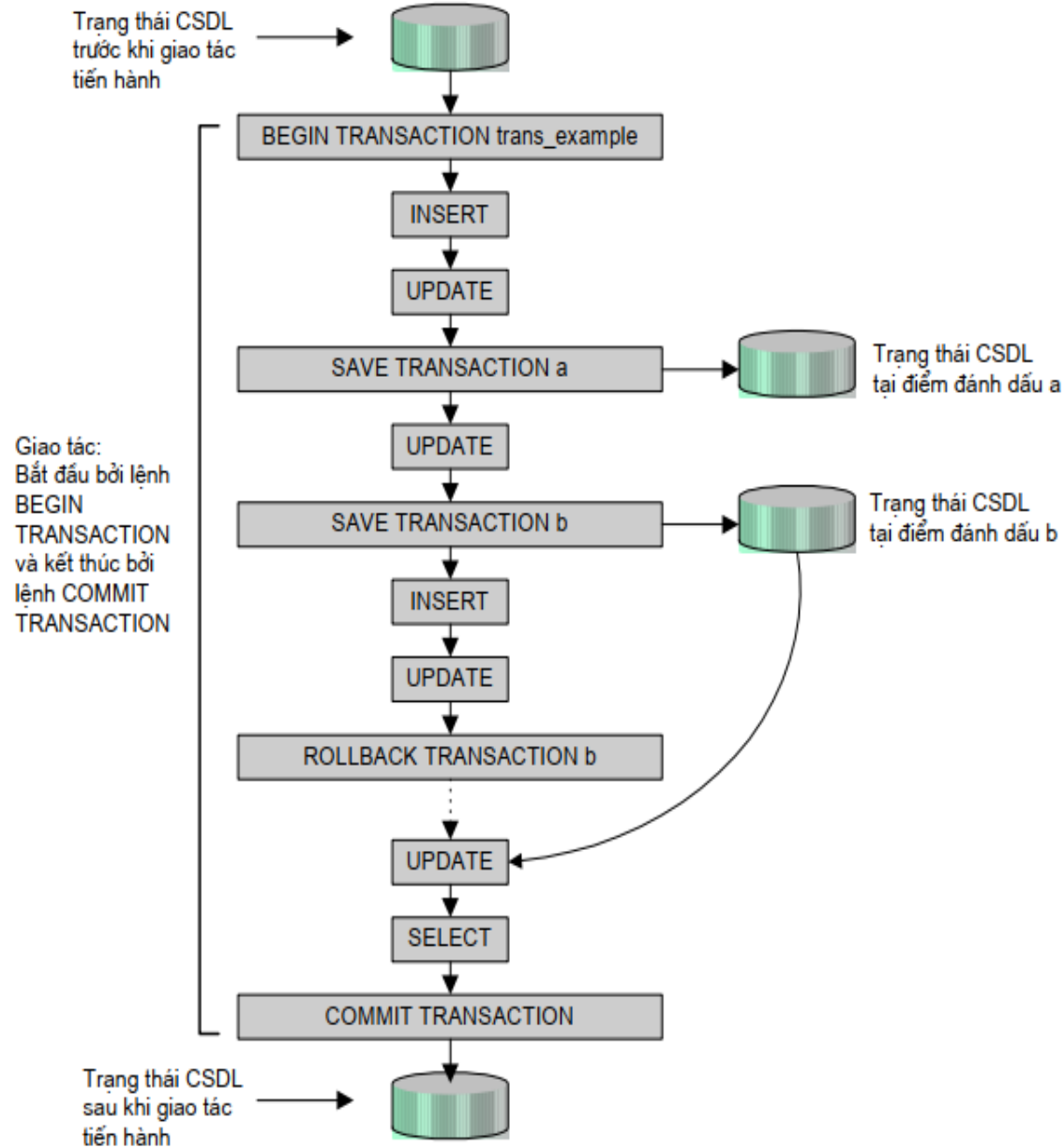
BEGIN TRANSACTION GIAOTAC1

UPDATE monhoc SET sodvht=5 WHERE sodvht=3

UPDATE diemthi SET diemlan2=0 WHERE diemlan2 IS NULL

COMMIT TRANSACTION GIAOTAC1

2. MÔ HÌNH GIAO TÁC TRONG SQL



Hoạt động của một giao tác



3. TRUY XUẤT ĐỒNG THỜI

3.1. Lost Update (mất dữ liệu cập nhật)

- Giả sử hệ thống thực hiện P_1 và P_2 theo thứ tự sau:

A = 40	P₁	P₂
T ₁	Read(A)	
T ₂		Read(A)
T ₃	A:=A+10	
T ₄	Write(A)	
T ₅		A:=A+20
T ₆		Write(A)

A= 50

A= 60



3. TRUY XUẤT ĐỒNG THỜI

3.1. Lost Update (mất dữ liệu cập nhật)

- Ta thấy:
 - Tại thời điểm T_4 P_1 ghi xuống giá trị là 50.
 - Nhưng sau đó, tại thời điểm T_6 P_2 ghi đè giá trị là 60 lên P_1 nên khi tại thời điểm T_4 giá trị bị mất.
 - Trường hợp này gọi là ***mất mát dữ liệu đã cập nhật***.
 - Tình trạng xảy ra khi hai hay nhiều thao tác của các giao tác khác nhau cùng yêu cầu truy cập một mục dữ liệu.
 - Các dữ liệu đã được các thao tác trước cập nhật nhưng lại bị các thao tác sau cập nhật lại làm thay đổi kết quả.



3. TRUY XUẤT ĐỒNG THỜI

3.2. Đọc phải dữ liệu rác (Dirty Read)

- P_1 và P_2 xử lý đồng thời:

A=40	P_1	P_2
T_1	Read(A)	
T_2	$A:=A+10$	
T_3	Write(A)	
T_4		Read(A)
T_5		Print(A)
T_6	Rollback	

A= 40

A= 50



3. TRUY XUẤT ĐỒNG THỜI

3.2. Đọc phải dữ liệu rác (Dirty Read)

- Xét hai giao tác P_1 và P_2 xử lý đồng thời:
 - P_2 đã đọc dữ liệu được ghi bởi P_1 nhưng sau đó P_1 yêu cầu hủy việc ghi dẫn đến dữ liệu không chính xác.
- Vấn đề trên được gọi là *đọc phải dữ liệu rác*.



3. TRUY XUẤT ĐỒNG THỜI

3.3. Không thể đọc lại (Unrepeatable Read)

- Giả sử hệ thống thực hiện P_1 và P_2 theo thứ tự sau:

A=40	P_1	P_2	
T_1	Read(A)		
T_2		Read(A)	A=40
T_3	$A:=A+10$		
T_4		Print(A)	A=40
T_5	Write(A)		
T_6		Read(A)	A=50
T_7		Print(A)	A=50



3. TRUY XUẤT ĐỒNG THỜI

3.3. Không thể đọc lại (Unrepeatable Read)

- P_2 đọc A hai lần cho ra kết quả khác nhau.
- Trong một giao tác mà các lần đọc cùng đơn vị dữ liệu cho kết quả khác nhau, trường hợp này gọi là *lỗi không đọc lại được dữ liệu*.

3. TRUY XUẤT ĐỒNG THỜI

3.4. Bóng ma dữ liệu (Phantom)

- Xét 2 giao tác P_1 và P_2 được xử lý đồng thời

A=60, B=40	P_1	P_2	
T_1	Read(A)		A=60
T_2	A:=A-50		
T_3	Write(A)		A=10
T_4		Read(A)	A=10
T_5		Read(B)	B=40
T_6		Print(A+B)	A+B=50 (mất 50)
T_7	Read(B)		
T_8	B:=B+50		
T_9	Write(B)		



3. TRUY XUẤT ĐỒNG THỜI

3.4. Bóng ma dữ liệu (Phantom)

- Xét 2 giao tác P_1 và P_2 được xử lý đồng thời
- A và B là hai tài khoản
- P_1 rút một số tiền ở tài khoản A rồi đưa vào tài khoản B
- P_2 kiểm tra đã nhận đủ tiền chưa?
 - Ở đây mỗi giao tác độc lập với nhau
 - Do Print(A+B) để ở thời điểm T_6 nên $A + B$ chỉ có 50 thiếu 50.
 - Nếu chúng ta để ở thời điểm cuối cùng thì $A + B = 100$
- Trường hợp như vậy gọi là ***bóng ma dữ liệu***.



3. TRUY XUẤT ĐỒNG THỜI

3.5. Cách giải quyết

- Các HQT CSDL thương mại đã giải quyết, ở đây chúng ta tìm hiểu lý thuyết để hiểu cách chúng làm việc.
- Thực hiện cơ chế transaction và cơ chế khóa.



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

- Một giao dịch P trước khi muốn thao tác (read/write) lên một đơn vị dữ liệu A phải phát ra một yêu cầu xin khóa A: **lock(A)**
- Nếu yêu cầu được chấp thuận thì giao dịch P mới được phép thao tác lên đơn vị dữ liệu A.
- Sau khi thao tác xong, giao dịch P phải phát ra lệnh giải phóng A: **unlock(A).**



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

Các kiểu lock

- Binary lock
- Shared lock
- Exclusive lock
- Intend to write lock (khóa dự định ghi)



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6. 1. Binary lock

- Có 2 trạng thái: locked(1) hoặc unlocked(0).
- Nếu 1 object bị lock bởi 1 transaction, không transaction nào được sử dụng object đó.
- Nếu 1 object là unlock bởi 1 transaction, transaction nào cũng sử dụng được đối tượng đó.
- 1 transaction phải “unlock” object sau khi hoàn tất.

3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6. 1. Binary lock

- Ví dụ:

S

T1	T2
Lock(A) Read(A, t) t := t + 100 Write(A, t) Unlock(A)	Lock(A) Read(A, s) s := s * 2 Write(A, s) Unlock(A) Lock(B) Read(B, s) s := s * 2 Write(B, s) Unlock(B)
Lock(B) Read(B, t) t := t + 100 Write(B, t) Unlock(B)	



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6. 1. Binary lock

- Giao tác muốn Write(A)
 - Yêu cầu WLock(A)
 - WLock(A) sẽ được chấp nhận nếu A tự do
 - ✓ Sẽ không có giao tác nào nhận được WLock(A) hay RLock(A)



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6. 1. Binary lock

- Giao tác muốn Read(A)
 - Yêu cầu RLock(A) hoặc WLock(A)
 - RLock(A) sẽ được chấp nhận nếu A không giữ một WLock nào.
 - ✓ Không ngăn chặn các thao tác khác cùng xin RLock(A)
 - ✓ Các giao tác không cần chờ nhau khi đọc.



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6.2 Shared lock

- Giao dịch giữ SLock được phép **ĐỌC** dữ liệu, nhưng không được phép **GHI**.
- *Nhiều giao dịch có thể đồng thời giữ SLock* trên cùng 1 đơn vị dữ liệu.



3. TRUY XUẤT ĐỒNG THỜI

3.6. Kỹ thuật khóa

3.6.3 Exclusive lock (Write Lock)

- Giao dịch giữ XLock được phép **GHI + ĐỌC** dữ liệu
- Tại 1 thời điểm chỉ có tối đa **1 giao dịch được quyền giữ XLock** trên 1 đơn vị dữ liệu.
- Không thể thiết lập SLock trên đơn vị dữ liệu đang có dạng XLock.



3. TRUY XUẤT ĐỒNG THỜI

3.6.4 Intend to write lock (khóa dự định ghi)

- **Update Lock = Intent - to - update Lock (ULock): Khóa dự định ghi**
 - ULock sử dụng khi đọc dữ liệu với dự định ghi trở lại trên dữ liệu này.
 - ULock là chế độ khoá trung gian giữa SLock và XLock.
 - Khi thực hiện thao tác **GHI** lên dữ liệu thì bắt buộc ULock phải tự động chuyển thành XLock.
 - Giao dịch giữ ULock được phép **GHI + ĐỌC** dữ liệu.
 - Tại 1 thời điểm chỉ có tối đa **1 giao dịch được quyền giữ ULock** trên 1 đơn vị dữ liệu.
 - Có thể thiết lập SLock trên đơn vị dữ liệu đang có dạng ULock.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch (Isolation)

- **Mục đích:**
 - Tự động đặt khóa cho các thao tác (đọc) trong kết nối dữ liệu hiện hành.
- **Các mức độ cô lập:**
 - Read Uncommitted
 - Read Committed
 - Repeatable Read
 - Serializable



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.1. Read Uncommitted

- **Đặc điểm:**
 - Đây là mức thấp nhất, khi có nhiều transaction diễn ra cùng lúc, data chưa commit của transaction này có thể được nhìn thấy bởi transaction khác nên data đọc được có thể bị rollback.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.1. Read Uncommitted

- **Ví dụ:** tạo một bảng như sau:

```
create table Emp(ID int,Name Varchar(50),Salary Int)
```

```
insert into Emp(ID,Name,Salary)  
values( 1,'David',1000)
```

```
insert into Emp(ID,Name,Salary)  
values( 2,'Steve',2000)
```

```
insert into Emp(ID,Name,Salary)  
values( 3,'Chris',3000)
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.1. Read Uncommitted

- **Ví dụ:**

- Ta chạy câu lệnh SQL trên hai session khác nhau
- Session 1

```
begin tran  
update emp set Salary=999 where ID=1  
waitfor delay '00:00:15'  
rollback
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.1. Read Uncommitted

- Ví dụ:

- Ta chạy câu lệnh SQL trên hai session khác nhau
- Session 2

```
set transaction isolation level read uncommitted  
select Salary from Emp where ID=1
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.1. Read Uncommitted

- **Kết quả:**
 - Truy vấn chọn trong Session2 thực thi sau khi cập nhật bảng Emp trong giao dịch và trước khi giao dịch được khôi phục. Do đó, **999** được trả về thay vì **1000**.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.2. Read Committed

- Chỉ có những data đã commit mới có thể nhìn thấy bởi transaction khác.
- Nhưng trong transaction, khi chạy cùng 1 lệnh **SELECT** ở những thời điểm khác nhau, data nhận được có thể không giống nhau do các transaction khác thay đổi. Hiện tượng này gọi là **non-consistent read**.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.2. Read Committed

- **Ví dụ:** tạo một bảng như sau

```
create table Emp(ID int,Name Varchar(50),Salary Int)
```

```
insert into Emp(ID,Name,Salary)  
values( 1,'David',1000)
```

```
insert into Emp(ID,Name,Salary)  
values( 2,'Steve',2000)
```

```
insert into Emp(ID,Name,Salary)  
values( 3,'Chris',3000)
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.2. Read Committed

- **Ví dụ:**

- Ta chạy câu lệnh SQL trên hai session khác nhau
- Session 1

```
begin tran  
update emp set Salary=999 where ID=1  
waitfor delay '00:00:15'  
commit
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.2. Read Committed

- Ví dụ:
 - Ta chạy câu lệnh SQL trên hai session khác nhau
 - Session 2

```
set transaction isolation level read committed  
select Salary from Emp where ID=1
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.2. Read Committed

- **Kết quả:**
 - Trong phiên thứ hai, nó chỉ trả về kết quả sau khi thực hiện xong toàn bộ giao dịch trong phiên đầu tiên do khóa đọc trên bảng **Emp**. Chúng tôi đã sử dụng lệnh chờ để trì hoãn 15 giây sau khi cập nhật bảng Emp trong giao dịch. Do đó, **999** được trả về là đúng.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- **Đặc điểm:**
 - Trong một transaction, khi lệnh SELECT đầu tiên được chạy, DB sẽ tạo ra 1 snapshot của toàn bộ data đã commit ở thời điểm đó.
 - Các transaction khác vẫn có thể **INSERT, UPDATE, DELETE** data nhưng trong transaction này sẽ không nhìn thấy được những thay đổi đó cho dù thay đổi đó đã commit.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- **Ưu điểm:**

- Giải quyết được 3 vấn đề: **Lost Updated**, **Dirty Read** và **Unrepeatable Read**

- **Khuyết điểm:**

- Chưa giải quyết được vấn đề **Phantom**, do vẫn cho phép **INSERT** những dòng dữ liệu thỏa điều kiện thiết lập SLock
- SLock được giữ đến hết giao dịch => cản trở việc cập nhật (UPDATE) dữ liệu của các giao dịch khác.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- **Ví dụ:** tạo một bảng như sau

```
create table Emp(ID int,Name Varchar(50),Salary Int)
```

```
insert into Emp(ID,Name,Salary)  
values( 1,'David',1000)
```

```
insert into Emp(ID,Name,Salary)  
values( 2,'Steve',2000)
```

```
insert into Emp(ID,Name,Salary)  
values( 3,'Chris',3000)
```




3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- Ví dụ:

- Ta chạy câu lệnh SQL trên hai session khác nhau
- Session 1

```
set transaction isolation level repeatable read
begin tran
select * from emp where ID in(1,2)
waitfor delay '00:00:15'
select * from Emp where ID in (1,2)
rollback
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- Ví dụ:
 - Ta chạy câu lệnh SQL trên hai session khác nhau
 - Session 2

```
update emp set Salary=999 where ID=1
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.3. Repeatable Read

- **Kết quả:**

- Dữ liệu truy vấn chọn (SELECT) của bảng được sử dụng trong giao dịch ở mức cô lập "**Đọc lặp lại**" không thể được sửa đổi (UPDATE) từ bất kỳ phiên nào khác cho đến khi quá trình chuyển đổi hoàn tất.
- Lệnh cập nhật trong phiên 2 sẽ đợi cho đến khi giao dịch phiên 1 hoàn tất vì hàng của bảng Emp có ID = 1 đã bị khóa trong giao dịch phiên 1.



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- **Đặc điểm:**
 - **Đọc dữ liệu:** SQL server tự động thiết lập SLock trên đơn vị dữ liệu được đọc và giữ SLock này đến hết giao dịch.
 - Không cho phép thêm (INSERT) những dòng dữ liệu thỏa mãn điều kiện thiết lập SLock.
 - **Ghi dữ liệu:** SQL server tự động thiết lập XLock trên đơn vị dữ liệu được ghi, XLock được giữ cho đến hết giao dịch



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- **Ví dụ:** tạo một bảng như sau

```
create table Emp(ID int,Name Varchar(50),Salary Int)
```

```
insert into Emp(ID,Name,Salary)  
values( 1,'David',1000)
```

```
insert into Emp(ID,Name,Salary)  
values( 2,'Steve',2000)
```

```
insert into Emp(ID,Name,Salary)  
values( 3,'Chris',3000)
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- Ví dụ:

- Ta chạy câu lệnh SQL trên hai session khác nhau
- Session 1

```
set transaction isolation level serializable
begin tran
select * from emp
waitfor delay '00:00:15'
select * from Emp
rollback
```



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- Ví dụ:
 - Ta chạy câu lệnh SQL trên hai session khác nhau
 - Session 2

```
insert into Emp(ID,Name,Salary)
values( 11, 'Stewart',11000)
```

3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- Ví dụ:
 - Chạy cả hai phiên song song.
 - Kết quả trong phiên 1

Results		Messages	
	ID	Name	Salary
1	1	David	1000
2	2	Steve	2000
3	3	Chris	3000

	ID	Name	Salary
1	1	David	1000
2	2	Steve	2000
3	3	Chris	3000



3. TRUY XUẤT ĐỒNG THỜI

3.7. Mức độ cô lập của giao dịch

3.7.4. Serializable

- **Kết luận:**

- Bảng Emp đầy đủ sẽ bị khóa trong quá trình giao dịch trong phiên 1. Không giống như **Repeatable Read**, truy vấn chèn (INSERT) trong Phiên 2 sẽ đợi cho đến khi thực hiện phiên 1 hoàn tất. Do đó, đọc **Phantom** bị chặn và cả hai truy vấn trong phiên 1 sẽ hiển thị cùng một số hàng.

3. TRUY XUẤT ĐỒNG THỜI

3.8. Deadlock

- Khi xử lý đồng thời, không tránh khỏi việc transaction này phải chờ đợi transaction khác
- Nếu vì lý do gì đó mà hai transaction lại chờ lẫn nhau vĩnh viễn, không cái nào trong hai có thể hoàn thành được thì ta gọi đó là hiện tượng Dead Lock

T ₁	T ₂	Ghi chú
Lock(A)		<i>T₁ đợi T₂ <u>unlock(B)</u> trong khi T₂ lại đợi T₁ unlock(A) để tiếp tục thực hiện. Hai giao dịch này cứ chờ nhau và cả hai đều không chạy được.</i>
	Lock(B)	
Lock(B) (Chờ)		
	Lock(A) (chờ)	



3. TRUY XUẤT ĐỒNG THỜI

3.8. Deadlock

Xử lý Deadlock:

- SQL Server sẽ chọn 1 trong 2 transaction gây deadlock để hủy bỏ, khi đó transaction còn lại sẽ được tiếp tục thực hiện cho đến khi hoàn tất.
- Transaction bị chọn hủy bỏ là transaction mà SQL ước tính chi phí cho phần việc đã làm được ít hơn transaction còn lại.

3. TRUY XUẤT ĐỒNG THỜI

3.8. Deadlock

Ví dụ Deadlock:

T1	T2
<pre>BEGIN TRAN SET ANSI_WARNINGS ON UPDATE KhachHang with (XLOCK) SET TenKH = 'ABC' WHERE MAKH = 'KH001' WAITFOR DELAY '00:00:05' UPDATE KhachHang with (XLOCK) SET TenKH = 'XYZ' WHERE MAKH = 'KH002' COMMIT TRAN</pre>	<pre>BEGIN TRAN SET ANSI_WARNINGS ON UPDATE KhachHang with (XLOCK) SET TenKH = '123' WHERE MAKH = 'KH002' UPDATE KhachHang with (XLOCK) SET TenKH = '456' WHERE MAKH = 'KH001' COMMIT TRAN</pre>



VANLANG UNIVERSITY
School of Technology
Information Technology

